

Dictionaries

CSCI 1101

2024-03-28

Lists as Look-up Tables

One way to think of a list:

```
beatles = ["John", "Paul", "George", "Ringo"]
```

is as a look-up table:

index		value
0	$\mapsto$	"John"
1	$\mapsto$	"Paul"
2	$\mapsto$	"George"
3	$\mapsto$	"Ringo"

From this perspective the indices are the numbers from 0 to the list length -1 .

But a table's look-up indices need not be these particular numbers.

Olympic men's 100m winners

index		value
0	$\mapsto$	"NA"
	$\vdots$	
1895	$\mapsto$	"NA"
1896	$\mapsto$	"Thomas Burke"
1897	$\mapsto$	"NA"
1898	$\mapsto$	"NA"
1899	$\mapsto$	"NA"
1900	$\mapsto$	"Frank Jarvis"
	$\vdots$	
2016	$\mapsto$	"Usain Bolt"
	$\vdots$	

... or even numbers at all.

Dictionaries

A data structure for look-up tables called a **dictionary**.

An index of a dictionary is called a **key**.

The item in a dictionary indexed by a key is called its **value**.

Dictionaries in Python

Python has a data structure for dictionaries, called `dict`.

For homogeneous dictionaries we will track the key and value types as well.

We write a dictionary as a sequence of `key:value` pairs between braces:

```
# pizza sizes: dict (str , int)
sizes = {"small":12 , "medium":16 , "large":20}
```

To convert any iterator of key-value pairs into a dictionary use the `dict` type converting function:

```
days = \
dict(enumerate(["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]))
```

In python the type of a dictionary's keys must be *immutable* (technically, *hashable*).

Key Assignment

To create an *empty dictionary* call the `dict` function without arguments:

```
olympics = dict() # dict (int , str)
```

To add a key-value pair use **key assignment**:

```
olympics[1896] = 'Thomas Burke'  
olympics[1988] = 'Ben Johnson'  
olympics[2008] = 'Usain Bolt'  
olympics[2012] = 'Usain Bolt'  
olympics[2016] = 'Usain Bolt'
```

You can also **update** the value associated with a key:

```
olympics[1988] = 'Carl Lewis' # doping
```

Dictionary Lookup

The main use of a dictionary is to look up the value associated to a key.

We do this with the **index operator** `_[_]`, just like for a list:

```
first_winner = olympics[1896]
```

It is an error to index out-of-bounds for a dictionary, just like for a list:

```
next_winner = olympics[2024]
```

The `dict.get` method also looks up the value associated to key, but takes an additional argument for the value to return if the key is invalid:

```
next_winner = olympics.get(2024 , "NA")
```

Getting Keys and Values

The `dict.keys` method returns a dictionary's keys:

```
years = olympics.keys()
```

The `dict.values` method returns all values stored under any key (with repetitions):

```
winners = olympics.values()
```

The `dict.items` method returns all of the key-value pairs in a dictionary:

```
records = olympics.items()
```

These are all iterator types, which you can `listify` if necessary.

Activity: Lookup All

Write a function

`lookup_all : (dict (a , b) , list a) --> list b`

that returns the list of all values associated with any of the argument keys.

If an argument list element is not a dictionary key then this should *not* cause an error, but no value should be added to the output list for this key.

For example:

```
> lookup_all(olympics , range(2008 , 2017))  
['Usain Bolt', 'Usain Bolt', 'Usain Bolt']
```

Removing Dictionary Entries

To remove an entry from a dictionary, use the `del` operator with the key that you want removed:

```
del sizes['large']      # sold out
```

It is an error to try to remove a dictionary entry for a non-existent key:

```
del sizes['extra-large']
```

You can have the corresponding value returned by using the `dict.pop` method instead:

```
sizes.pop('small')
```

Iterating Over a Dictionary

The most common way to iterate over a dictionary is using its keys:

```
for key in sorted(olympics.keys()) :  
    print (f"{key} : {olympics[key]}")
```

You can save the lookup step by iterating over key-value pairs:

```
for (key , val) in sorted(olympics.items()) :  
    print (f"{key} : {val}")
```

You can iterate over just the values, but then you lose the key-value association:

```
for val in sorted(olympics.values()) :  
    print (f"{val} won at some point")
```

Activity: Removing all Keys for a Given Value

Write a function

```
remove_all : (dict (a , b) , b) --> list a
```

that takes as arguments a dictionary and an element of its value type, modifies the dictionary by removing all entries whose value matches the one given, and returns the list of removed keys.

For example:

```
> remove_all(olympics , "Usain Bolt")  
[2008, 2012, 2016]
```

Dictionary Reverse Lookup

Sometimes we want to look something up in a dictionary, but we know the *value* rather than the *key*.

There is no efficient way to do this, we just need to iterate over the entries until we find what we are looking for.

But it gets worse: while every key is associated with a single value, a given value may be associated with multiple keys.

Thus when we do reverse lookup by value, we should expect a *set* of keys.

Activity: Inverting a Dictionary

Write a function

```
invert_dict : (dict (a , b)) --> dict (b , set a)
```

so that the *keys* of the returned dictionary are the *values* of the argument dictionary and the *values* of the returned dictionary are the *sets of keys* associated with those values in the argument dictionary.

For example:

```
> invert_dict(olympics)
{'Thomas Burke': {1896}, 'Carl Lewis': {1988},
 'Usain Bolt': {2008, 2012, 2016}}
```

To Do

Finish *Homework 8: Sets and Dictionaries* on CodeRunner by 11:00AM next Thursday.

Finish *Project 3 - Automated Grader (Part 1)* on CodeRunner by midnight next Friday.